

Design and Verification of an 8-Bit Arithmetic Logic Unit (ALU) Using Verilog HDL

Technical Project Report

AI-Assisted RTL Design, Simulation, Verification, and Synthesis using AMD
Vivado

Author

Shivam Chaurasiya

B.Tech in Electronics and Communication Engineering

Project Type

Self-Directed Learning Project

Development Environment

Language	: Verilog HDL
EDA Tool	: AMD Vivado 2025.2
Verification	: AI-Assisted Testbench Generation
Target Design	: 8-Bit Arithmetic Logic Unit (ALU)

GitHub Repository:

<https://github.com/shivam-du/8-bit-alu-ai-verification>

July 2026

Contents

Abstract	v
1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Project Objectives	2
1.4 Functional Requirements	2
1.5 Block Diagram	2
1.6 Development Tools	3
2 Design Methodology	4
2.1 Design Overview	4
2.2 Design Flow	4
2.3 ALU Architecture	6
2.4 Input and Output Description	6
2.5 Supported Operations	7
2.6 RTL Implementation	7
2.7 Verilog HDL Implementation	7
2.8 Design Considerations	8
3 AI-Assisted Verification	9
3.1 Introduction	9
3.2 AI-Based Testbench Generation	9
3.3 Analysis of the AI-Generated Testbench	10
3.4 Missing Verification Scenarios	10
3.5 Prompt Refinement	11
3.6 Improved Verification Strategy	11
3.7 Advantages of AI-Assisted Verification	11
3.8 Verification Flow	12
4 Simulation and Verification	13
4.1 Introduction	13

4.2	Simulation Environment	13
4.3	Verification Using the AI-Generated Testbench	13
4.4	Improved Verification	14
4.5	Simulation Results	15
4.6	Edge Case Verification	15
4.7	Simulation Console Output	15
4.8	Verification Summary	16
5	RTL Synthesis	17
5.1	Introduction	17
5.2	Device View (Post-Synthesis)	17
5.3	Synthesized Netlist	19
5.4	Synthesis Resource Utilization	19
5.5	Power Analysis	20
5.6	Hardware Optimization	22
5.7	FPGA Resources	22
5.8	Discussion	22
5.9	Advantages of RTL Synthesis	23
6	Comparative Evaluation	24
6.1	Introduction	24
6.2	Comparison of Testbenches	24
6.3	Analysis of the AI-Generated Testbench	25
6.4	Manual Improvements	25
6.5	Prompt Refinement	26
6.6	Lessons Learned	26
6.7	Overall Evaluation	26
7	Conclusion and Future Scope	28
7.1	Conclusion	28
7.2	Future Scope	29
	References	30

List of Figures

1.1	Functional Block Diagram of the 8-Bit Arithmetic Logic Unit	3
2.1	Overall Design Flow of the Project	5
2.2	RTL Schematic of the Designed 8-Bit ALU	6
3.1	AI-Assisted Verification Workflow	12
4.1	Behavioral Simulation using the AI-Generated Testbench	14
4.2	Behavioral Simulation using the Improved Testbench	14
4.3	Simulation Console Output Generated by XSim	16
5.1	Device View of the Synthesized 8-Bit ALU in AMD Vivado	18
5.2	Synthesized Gate-Level Netlist	19
5.3	Post-Synthesis Resource Utilization Report	20
5.4	Post-Synthesis Power Estimation Report	21

List of Tables

1.1	Opcode Description of the Implemented ALU	2
1.2	Software Tools Used	3
2.1	Input and Output Description	6
2.2	Supported ALU Operations	7
3.1	Missing Test Cases in the AI-Generated Testbench	10
4.1	Simulation Environment	13
4.2	Verification Results for Mandatory Test Cases	15
4.3	Additional Verification Cases	15
5.1	Post-Synthesis Resource Utilization	20
5.2	FPGA Resources Utilized by the ALU	22
6.1	Comparison of Verification Approaches	24

Abstract

An Arithmetic Logic Unit (ALU) is one of the fundamental building blocks of every digital processor, responsible for performing arithmetic and logical operations required during data processing. The objective of this project was to design, verify, and synthesize an 8-bit Arithmetic Logic Unit (ALU) using Verilog Hardware Description Language (HDL). The implemented ALU performs eight different arithmetic and logical operations selected through a 3-bit opcode.

The design was developed using synthesizable Verilog HDL and simulated using AMD Vivado 2025.2. Functional verification was carried out using a comprehensive Verilog testbench generated through Artificial Intelligence (AI) assistance and subsequently refined through manual verification. In addition to the required functional test cases, several boundary conditions, overflow conditions, logical operations, and random input vectors were incorporated to improve verification coverage and ensure design correctness.

Behavioral simulation confirmed the correct functionality of all ALU operations. The design was then synthesized successfully using AMD Vivado, demonstrating that the RTL description can be efficiently implemented in FPGA hardware. The synthesized netlist and RTL schematic further validate the correctness of the hardware implementation.

This project demonstrates a complete digital design workflow including RTL development, AI-assisted verification, simulation, debugging, synthesis, and design validation. It also highlights the importance of critically evaluating AI-generated verification environments and improving them through systematic engineering analysis to achieve comprehensive functional verification.

CHAPTER 1

Introduction

1.1 Background

The Arithmetic Logic Unit (ALU) is one of the most fundamental components of every digital processor. It is responsible for performing arithmetic and logical operations on binary data during instruction execution. Almost every modern microprocessor, microcontroller, digital signal processor (DSP), and embedded computing system contains one or more ALUs that execute mathematical calculations and logical decision-making operations.

An ALU operates on binary operands supplied by processor registers and produces an output based on a control signal known as the operation code (opcode). Depending on the opcode, the ALU performs operations such as addition, subtraction, multiplication, logical AND, logical OR, logical XOR, and shift operations. The output generated by the ALU is subsequently stored in registers or forwarded to other processing units within the processor architecture.

Since the ALU directly influences processor performance, its design requires careful consideration of functionality, correctness, hardware complexity, and verification. Consequently, designing and verifying an ALU serves as an excellent introductory project for understanding digital system design using Hardware Description Languages (HDLs).

1.2 Motivation

Digital system design involves much more than writing synthesizable hardware description language (HDL) code. A complete design cycle includes functional design, simulation, verification, debugging, synthesis, and hardware implementation. Therefore, this project was undertaken as a self-directed learning exercise to gain practical experience in the complete FPGA-based digital design workflow.

Another important motivation behind this project was to explore the application of Artificial Intelligence (AI) during the verification phase. Rather than manually creating every verification test case, AI was employed to generate an initial Verilog testbench. The generated verification environment was then analyzed, improved, and validated through manual engineering judgment, thereby demonstrating the collaborative use of AI in modern hardware design workflows.

1.3 Project Objectives

The primary objectives of this project are:

- Design a synthesizable 8-bit Arithmetic Logic Unit using Verilog HDL.
- Implement eight arithmetic and logical operations selected through a 3-bit opcode.
- Generate an initial Verilog testbench using Artificial Intelligence.
- Analyze the AI-generated verification environment.
- Improve the verification coverage by introducing boundary conditions and edge cases.
- Validate the functionality of the ALU through behavioral simulation using AMD Vivado.
- Synthesize the design and analyze the generated hardware architecture.

1.4 Functional Requirements

The ALU accepts two 8-bit operands and a 3-bit operation code. Depending upon the opcode value, one of the predefined arithmetic or logical operations is executed. The result is produced on a 16-bit output bus along with a carry output wherever applicable.

The implemented ALU supports the operations listed in Table 1.1.

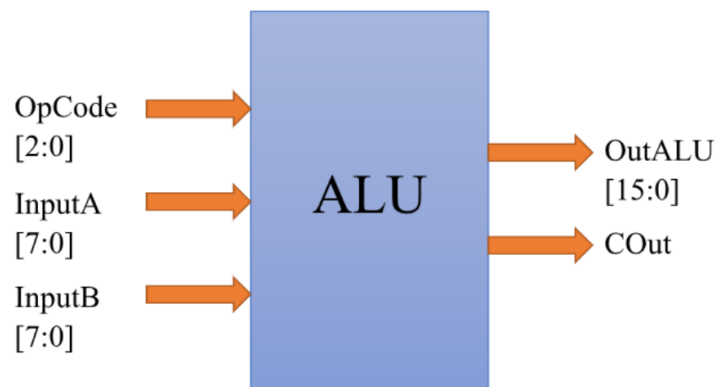
Table 1.1: Opcode Description of the Implemented ALU

Opcode	Operation
000	Addition
001	Subtraction
010	Multiplication
011	Left Shift
100	Right Shift
101	Logical AND
110	Logical OR
111	Logical XOR

1.5 Block Diagram

Figure 1.1 illustrates the functional architecture of the implemented Arithmetic Logic Unit. Two 8-bit input operands are processed according to the selected opcode, and the corresponding arithmetic or logical result is generated at the output.

Block Diagram:

**Figure 1.1:** Functional Block Diagram of the 8-Bit Arithmetic Logic Unit

1.6 Development Tools

The complete design, verification, and synthesis workflow was carried out using the following software tools.

Table 1.2: Software Tools Used

Tool	Purpose
Verilog HDL	RTL Design
AMD Vivado 2025.2	Simulation and Synthesis
ChatGPT	AI-Assisted Testbench Generation
GitHub	Project Documentation and Version Control

CHAPTER 2

Design Methodology

2.1 Design Overview

The objective of this project was to design an 8-bit Arithmetic Logic Unit (ALU) capable of performing eight arithmetic and logical operations. The ALU was described using synthesizable Verilog Hardware Description Language (HDL) and implemented as a purely combinational digital circuit. The selection of the required operation is controlled by a 3-bit opcode input.

The complete design follows the Register Transfer Level (RTL) design methodology. Each operation is implemented independently, while a `case` statement is used to select the desired output according to the opcode. The design avoids the use of non-synthesizable constructs to ensure compatibility with FPGA synthesis tools.

2.2 Design Flow

The complete development procedure adopted during this project is illustrated in Figure [2.1](#).

The development process consists of the following steps:

1. Requirement Analysis
2. RTL Design using Verilog HDL
3. AI-Assisted Testbench Generation
4. Functional Simulation
5. Verification and Debugging
6. RTL Synthesis
7. Design Analysis

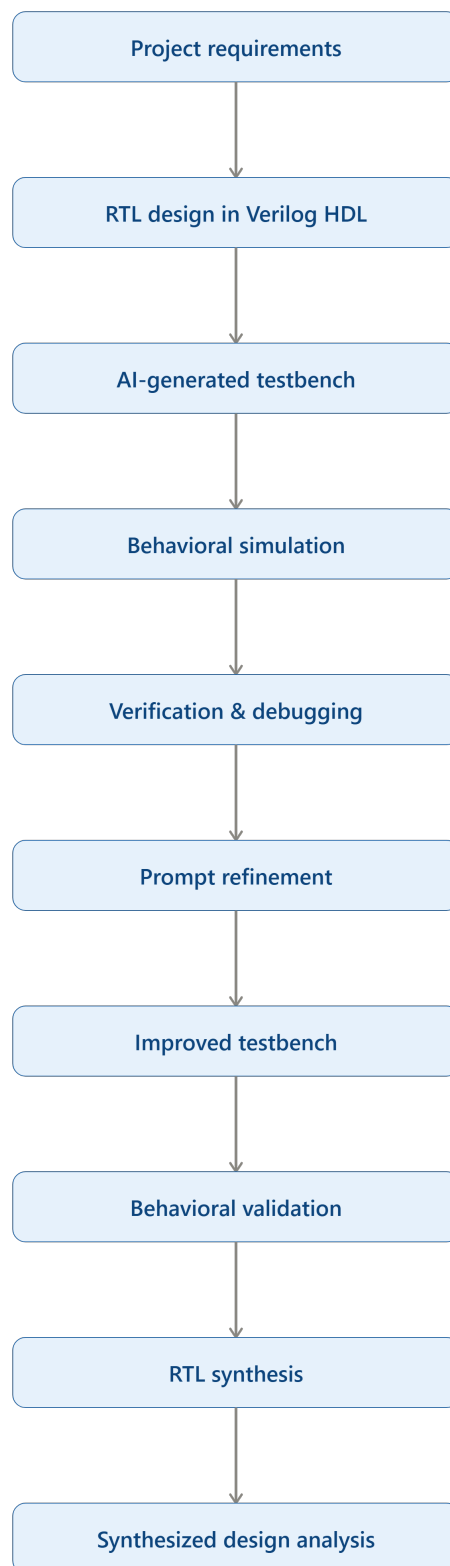


Figure 2.1: Overall Design Flow of the Project

2.3 ALU Architecture

The implemented ALU consists of two 8-bit input operands, one 3-bit opcode input, a 16-bit output bus, and a carry output. Depending upon the opcode value, one of the arithmetic or logical operations is executed.

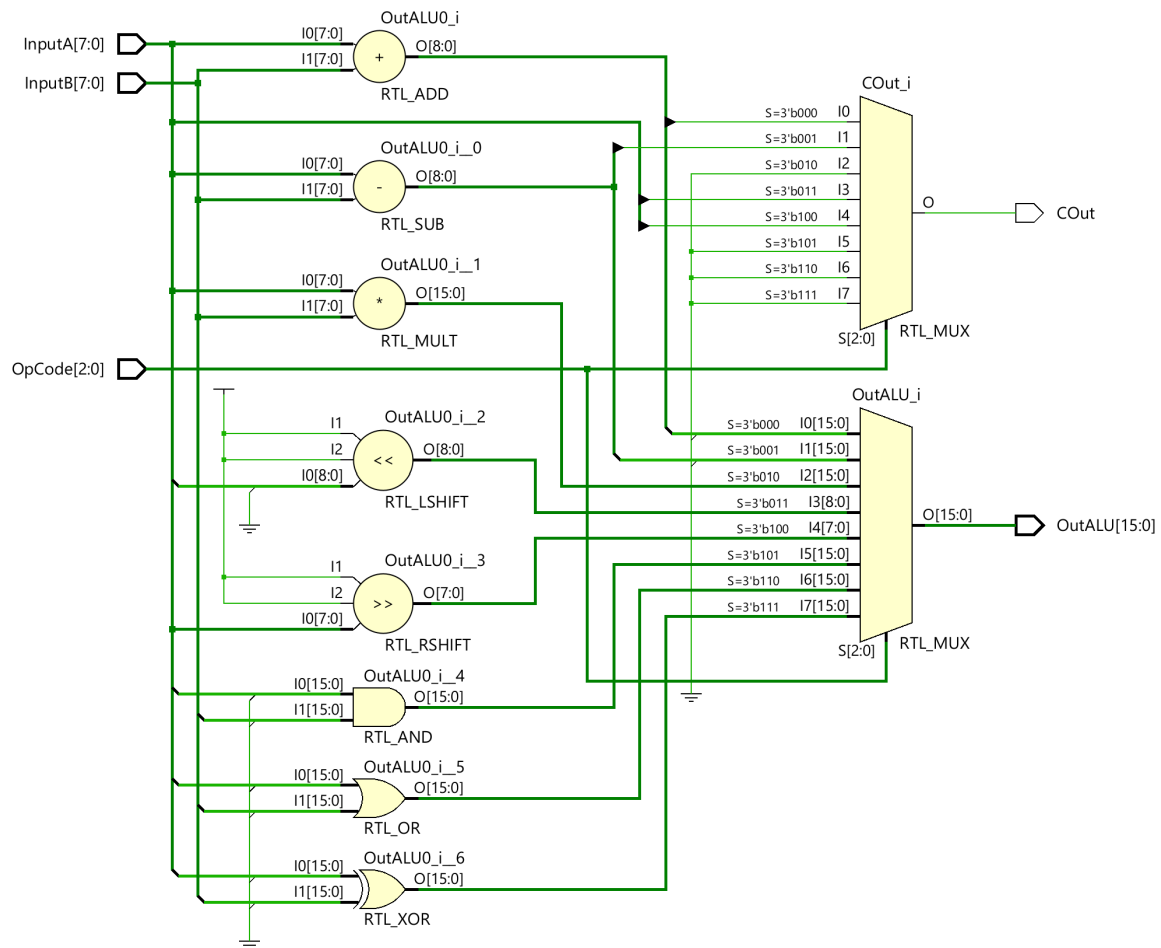


Figure 2.2: RTL Schematic of the Designed 8-Bit ALU

2.4 Input and Output Description

The input and output ports of the ALU are summarized in Table 2.1.

Table 2.1: Input and Output Description

Signal	Width	Type	Description
InputA	8-bit	Input	First Operand
InputB	8-bit	Input	Second Operand
OpCode	3-bit	Input	Operation Selector
OutALU	16-bit	Output	ALU Result
COut	1-bit	Output	Carry Output (when applicable)

2.5 Supported Operations

The ALU performs eight operations depending upon the opcode supplied to the circuit. Arithmetic operations produce numerical results, whereas logical operations perform bitwise manipulation of the input operands.

Table 2.2 summarizes the implemented functionality.

Table 2.2: Supported ALU Operations

Opcode	Operation	Expression
000	Addition	$A + B$
001	Subtraction	$A - B$
010	Multiplication	$A \times B$
011	Left Shift	$A \ll 1$
100	Right Shift	$A \gg 1$
101	Logical AND	$A \& B$
110	Logical OR	$A B$
111	Logical XOR	$A \oplus B$

2.6 RTL Implementation

The ALU was implemented using synthesizable Verilog HDL. A combinational `always @(*)` block together with a `case` statement was employed to implement opcode-based operation selection. Default assignments were provided for both `OutALU` and `COut` to prevent latch inference during synthesis.

For arithmetic operations, carry information was generated whenever applicable. Multiplication utilized the complete 16-bit output bus to accommodate the maximum product of two 8-bit operands. Logical operations performed bitwise manipulation without affecting the carry output.

2.7 Verilog HDL Implementation

The complete RTL source code is provided in Appendix A. Listing ?? presents the module declaration of the implemented ALU.

The complete synthesizable Verilog HDL implementation of the proposed 8-bit Arithmetic Logic Unit is provided in Appendix ?. The RTL code follows a combinational design methodology using a `case` statement to select the required arithmetic or logical operation based on the 3-bit opcode.

2.8 Design Considerations

During the implementation of the ALU, the following design considerations were adopted:

- Only synthesizable Verilog constructs were used.
- A combinational design methodology was adopted.
- Default assignments were included to avoid unintended latch inference.
- Output width was selected as 16 bits to accommodate multiplication results.
- Carry output was generated only for operations where it was meaningful.
- The design was written to ensure compatibility with AMD Vivado synthesis tools.

CHAPTER 3

AI-Assisted Verification

3.1 Introduction

Verification is one of the most critical phases in the digital design workflow. Although an RTL design may compile successfully, it must be thoroughly verified to ensure that it performs the intended functionality under different operating conditions. In this project, Artificial Intelligence (AI) was incorporated into the verification process to automatically generate an initial Verilog testbench. The generated testbench was then analyzed, refined, and validated to improve verification coverage and overall design reliability.

Unlike traditional verification, where every test case is manually developed, AI-assisted verification significantly reduces development time by generating an initial verification environment. However, engineering judgment remains essential to identify missing scenarios, validate the generated outputs, and improve test coverage.

3.2 AI-Based Testbench Generation

An AI language model was used to generate the initial Verilog testbench based on a structured prompt describing the ALU interface and the required functional test cases.

The following prompt was provided to the AI model.

Generate a Verilog testbench for an 8-bit ALU with ports InputA [7:0], InputB [7:0], OpCode [2:0], OutALU [15:0], and COut. Apply InputA = 45 and InputB = 38. Test opcode values 000, 001, 110, and 111. Then apply InputA = 49 and InputB = 10. Test opcode values 010, 011, 100, and 101. Display outputs using \$monitor or \$display.

The generated testbench correctly instantiated the ALU module, initialized the required inputs, applied the specified opcode combinations, and displayed simulation outputs using Verilog system tasks.

3.3 Analysis of the AI-Generated Testbench

The AI-generated testbench was examined to evaluate its completeness and verification capability. The analysis revealed that while the generated verification environment correctly executed the mandatory functional test cases, it lacked sufficient verification coverage for corner cases and boundary conditions.

The AI-generated testbench successfully verified:

- Addition
- Subtraction
- Multiplication
- Left Shift
- Right Shift
- Logical AND
- Logical OR
- Logical XOR

Although these operations confirmed the basic functionality of the ALU, several important verification scenarios were absent.

3.4 Missing Verification Scenarios

The following edge cases were not included in the initial AI-generated testbench.

Table 3.1: Missing Test Cases in the AI-Generated Testbench

Missing Case	Purpose
Zero Inputs	Verify operation with zero operands
Addition Overflow	Verify carry generation
Maximum Multiplication	Verify 16-bit output range
Equal Operands	Verify subtraction resulting in zero
Left Shift Boundary	Verify MSB carry-out
Right Shift Boundary	Verify LSB carry-out
Random Test Vectors	Improve verification coverage

3.5 Prompt Refinement

To improve verification quality, the original AI prompt was refined by explicitly requesting boundary conditions, overflow scenarios, logical operation testing, and random test vectors.

The refined prompt is shown below.

Generate a Verilog testbench for an 8-bit ALU with ports InputA [7:0], InputB [7:0], OpCode [2:0], OutALU [15:0], and COut. Apply the required functional test cases and additionally verify zero inputs, addition overflow, subtraction resulting in zero, multiplication overflow, left shift boundary, right shift boundary, logical operations with maximum values, and random test vectors. Display outputs using both \$display and \$monitor, include simulation timestamps, and terminate the simulation using \$finish.

The refined prompt enabled the AI to generate a significantly more comprehensive verification environment.

3.6 Improved Verification Strategy

The improved testbench incorporated several additional verification scenarios beyond the assignment requirements. These included overflow testing, logical boundary conditions, maximum operand values, subtraction resulting in zero, and randomized verification vectors.

The additional test cases substantially increased functional verification coverage and provided greater confidence in the correctness of the ALU implementation.

3.7 Advantages of AI-Assisted Verification

The use of Artificial Intelligence during verification offered several practical advantages.

- Faster generation of the initial testbench.
- Reduced manual coding effort.
- Improved productivity during verification.
- Easier generation of repetitive test vectors.
- Rapid modification through prompt refinement.
- Better understanding of verification planning.

Despite these advantages, the AI-generated testbench still required manual review to ensure complete functional coverage.

3.8 Verification Flow

Figure 3.1 illustrates the AI-assisted verification methodology adopted during this project.

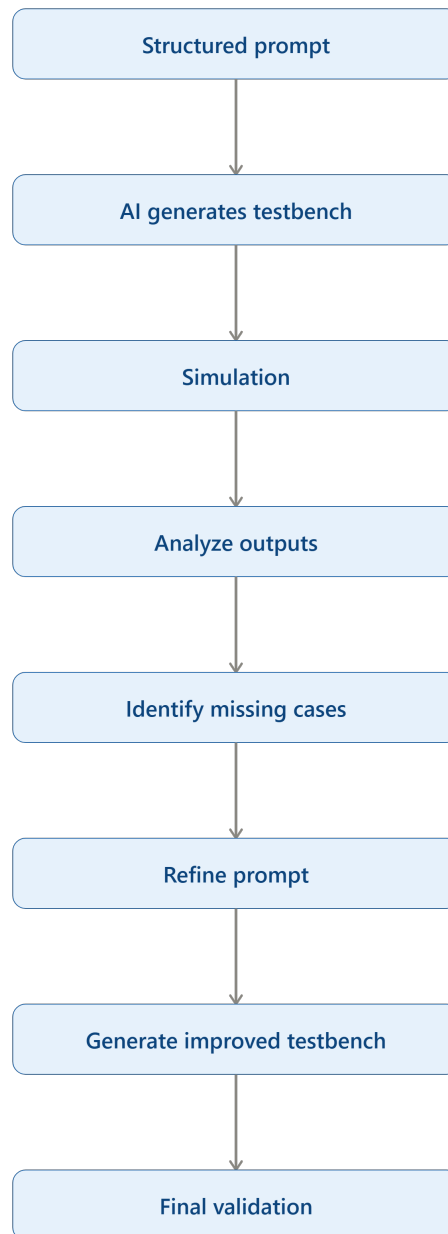


Figure 3.1: AI-Assisted Verification Workflow

CHAPTER 4

Simulation and Verification

4.1 Introduction

Functional verification is an essential stage of digital system design that ensures the implemented hardware behaves according to the specified functional requirements. After completing the RTL design of the 8-bit Arithmetic Logic Unit (ALU), behavioral simulation was carried out using AMD Vivado 2025.2. The primary objective of simulation was to validate the correctness of each arithmetic and logical operation before synthesis.

Initially, the ALU was verified using the AI-generated testbench containing the mandatory functional test cases. Subsequently, an improved testbench incorporating additional edge cases and random test vectors was developed to increase verification coverage.

4.2 Simulation Environment

Behavioral simulation was performed using the XSim simulator integrated within AMD Vivado 2025.2.

The simulation environment consisted of the following components:

Table 4.1: Simulation Environment

Parameter	Description
Design Language	Verilog HDL
Simulator	XSim
EDA Tool	AMD Vivado 2025.2
Simulation Type	Behavioral Simulation
Time Resolution	1 ps
Target Design	8-Bit Arithmetic Logic Unit

4.3 Verification Using the AI-Generated Testbench

The AI-generated testbench applied the mandatory input combinations specified in the project requirements. The simulation confirmed that all eight ALU operations produced the expected outputs.

Figure 4.1 shows the waveform obtained using the original AI-generated verification environment.

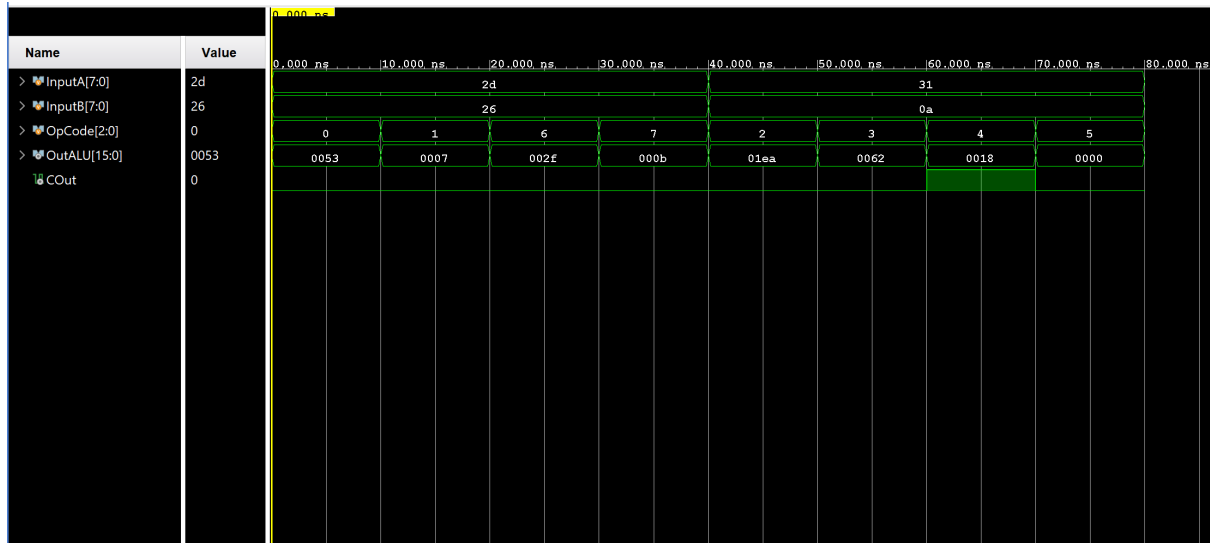


Figure 4.1: Behavioral Simulation using the AI-Generated Testbench

Although the waveform verified the required functionality, it did not include sufficient edge-case testing.

4.4 Improved Verification

To improve verification coverage, additional test vectors were introduced into the testbench. These included arithmetic overflow conditions, logical boundary cases, maximum-value operations, subtraction resulting in zero, and randomized input vectors.

Figure 4.2 illustrates the simulation waveform obtained using the improved verification environment.

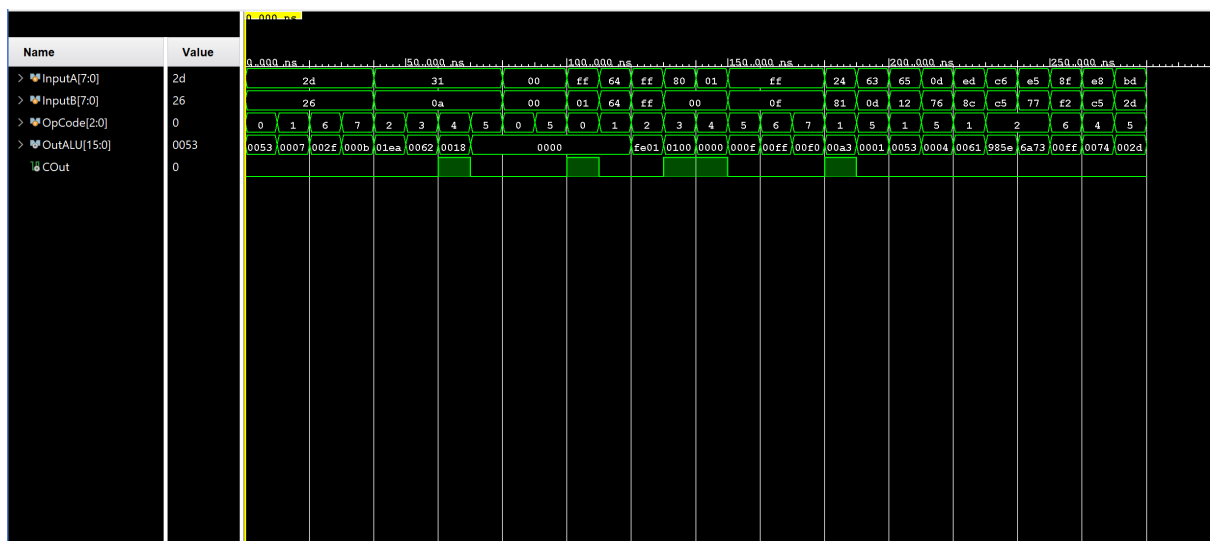


Figure 4.2: Behavioral Simulation using the Improved Testbench

The waveform demonstrates successful execution of all functional operations together with additional verification scenarios.

4.5 Simulation Results

Table 4.2 summarizes the outputs obtained during simulation for the mandatory functional test cases.

Table 4.2: Verification Results for Mandatory Test Cases

Input A	Input B	Opcode	Operation	Expected	Obtained
45	38	000	Addition	83	83
45	38	001	Subtraction	7	7
45	38	110	OR	47	47
45	38	111	XOR	11	11
49	10	010	Multiplication	490	490
49	10	011	Left Shift	98	98
49	10	100	Right Shift	24	24
49	10	101	AND	0	0

4.6 Edge Case Verification

The refined testbench further verified the functionality of the ALU using several important boundary conditions.

Table 4.3: Additional Verification Cases

Test Case	Expected Behaviour	Status
Zero Inputs	Output remains zero	Passed
Addition Overflow	Carry generated correctly	Passed
Equal Operands	Subtraction equals zero	Passed
Maximum Multiplication	16-bit product generated	Passed
Left Shift Boundary	Carry-out generated	Passed
Right Shift Boundary	Carry-out generated	Passed
Maximum Logical Operations	Correct logical outputs	Passed
Random Test Vectors	Functional correctness maintained	Passed

4.7 Simulation Console Output

The TCL Console generated by XSim displays the input operands, opcode, ALU output, carry output, and simulation timestamp for every test vector.

Figure 4.3 presents the simulation console output obtained during verification.

Time	InputA	InputB	Opcode	OutALU	COut
	0	45	38 000	83 (0x0053)	0
	10000	45	38 001	7 (0x0007)	0
	20000	45	38 110	47 (0x002f)	0
	30000	45	38 111	11 (0x000b)	0
	40000	49	10 010	490 (0x01ea)	0
	50000	49	10 011	98 (0x0062)	0
	60000	49	10 100	24 (0x0018)	1
	70000	49	10 101	0 (0x0000)	0
	80000	0	0 000	0 (0x0000)	0
	90000	0	0 101	0 (0x0000)	0
	100000	255	1 000	0 (0x0000)	1
	110000	100	100 001	0 (0x0000)	0
	120000	255	255 010	65025 (0xfe01)	0
	130000	128	0 011	256 (0x0100)	1
	140000	1	0 100	0 (0x0000)	1
	150000	255	15 101	15 (0x000f)	0
	160000	255	15 110	255 (0x00ff)	0
	170000	255	15 111	240 (0x00f0)	0
	180000	36	129 001	163 (0x00a3)	1
	190000	99	13 101	1 (0x0001)	0
	200000	101	18 001	83 (0x0053)	0
	210000	13	118 101	4 (0x0004)	0
	220000	237	140 001	97 (0x0061)	0
	230000	198	197 010	39006 (0x985e)	0
	240000	229	119 010	27251 (0x6a73)	0
	250000	143	242 110	255 (0x00ff)	0
	260000	232	197 100	116 (0x0074)	0
	270000	189	45 101	45 (0x002d)	0

Figure 4.3: Simulation Console Output Generated by XSim

The console output confirms that every expected output matches the corresponding simulated result.

4.8 Verification Summary

Behavioral simulation successfully verified all implemented ALU operations. The AI-generated testbench correctly validated the mandatory functionality, while the refined testbench significantly improved verification coverage by incorporating additional edge cases and random input vectors.

No functional mismatches were observed. The successful verification results provided confidence that the RTL implementation was suitable for synthesis and hardware realization.

CHAPTER 5

RTL Synthesis

5.1 Introduction

Following successful behavioral verification, the 8-bit Arithmetic Logic Unit (ALU) was synthesized using AMD Vivado 2025.2. RTL synthesis converts the hardware description written in Verilog HDL into a technology-specific gate-level representation suitable for FPGA implementation. This process verifies that the RTL description is synthesizable and generates an optimized hardware architecture consisting of FPGA primitives such as lookup tables (LUTs), multiplexers, carry chains, and input/output buffers.

Successful synthesis confirms that the designed ALU can be implemented on FPGA hardware without requiring modifications to the RTL description.

5.2 Device View (Post-Synthesis)

After successful synthesis, AMD Vivado maps the RTL description of the ALU onto the target FPGA architecture. The Device View provides a physical representation of how the synthesized logic is distributed across the FPGA fabric. Rather than showing the logical functionality, it illustrates the placement of the implemented design within the available FPGA resources.

The synthesized ALU occupies only a small portion of the available FPGA resources, indicating that the design is compact and hardware-efficient. The highlighted regions correspond to the slices utilized by the synthesis tool for implementing the combinational logic associated with arithmetic operations, logical operations, multiplexing circuitry, and output generation.

The device placement is automatically optimized by Vivado to ensure efficient routing while minimizing resource utilization. Since the ALU is purely combinational, the implementation requires only a limited number of configurable logic resources and does not consume significant FPGA area.

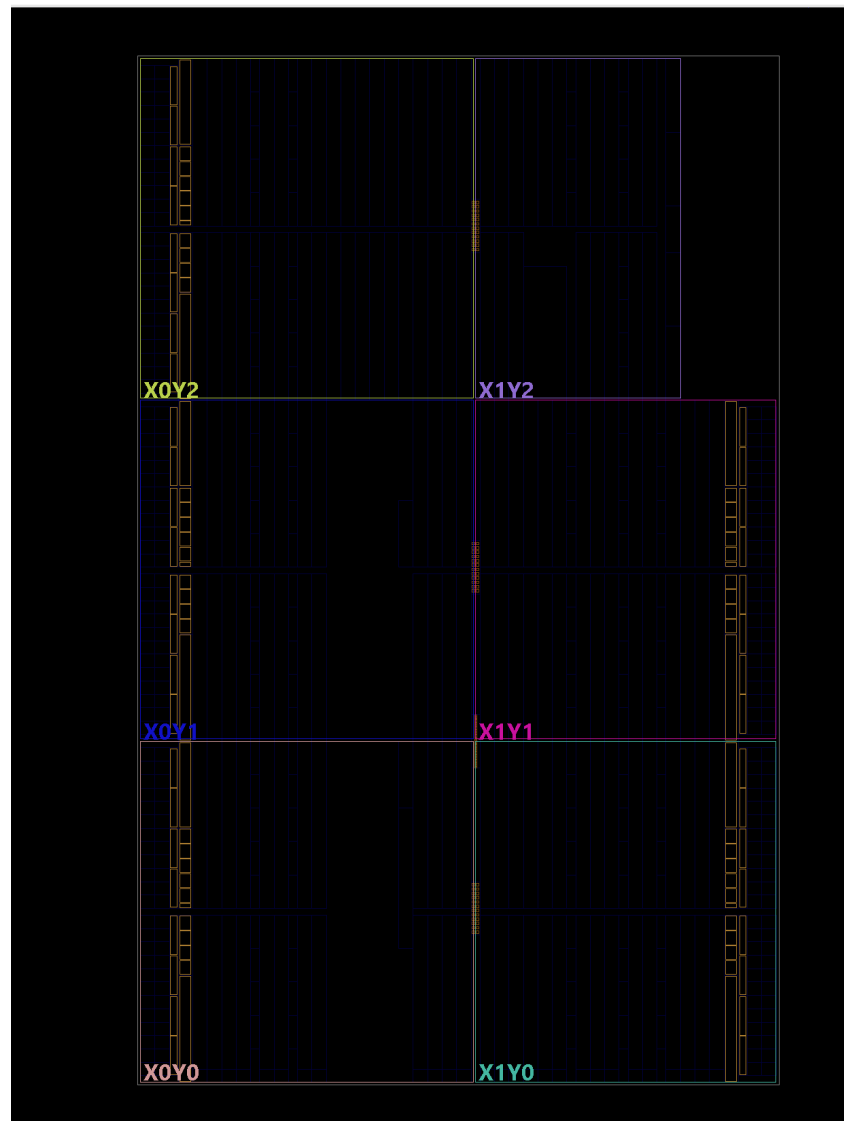


Figure 5.1: Device View of the Synthesized 8-Bit ALU in AMD Vivado

The following observations can be made from the device view:

- The synthesized ALU occupies only a small region of the FPGA fabric.
- Logic resources are automatically distributed by the synthesis tool to facilitate efficient routing.
- The placement demonstrates that the proposed design is lightweight and suitable for integration into larger FPGA-based systems.
- No routing congestion is observed, indicating an efficient hardware implementation.

5.3 Synthesized Netlist

After RTL synthesis, Vivado optimizes the design and converts it into FPGA-specific hardware resources. High-level arithmetic operators are translated into LUTs, multiplexers, carry-chain logic, routing resources, and input/output buffers.

Figure 5.2 presents the synthesized gate-level netlist of the ALU.

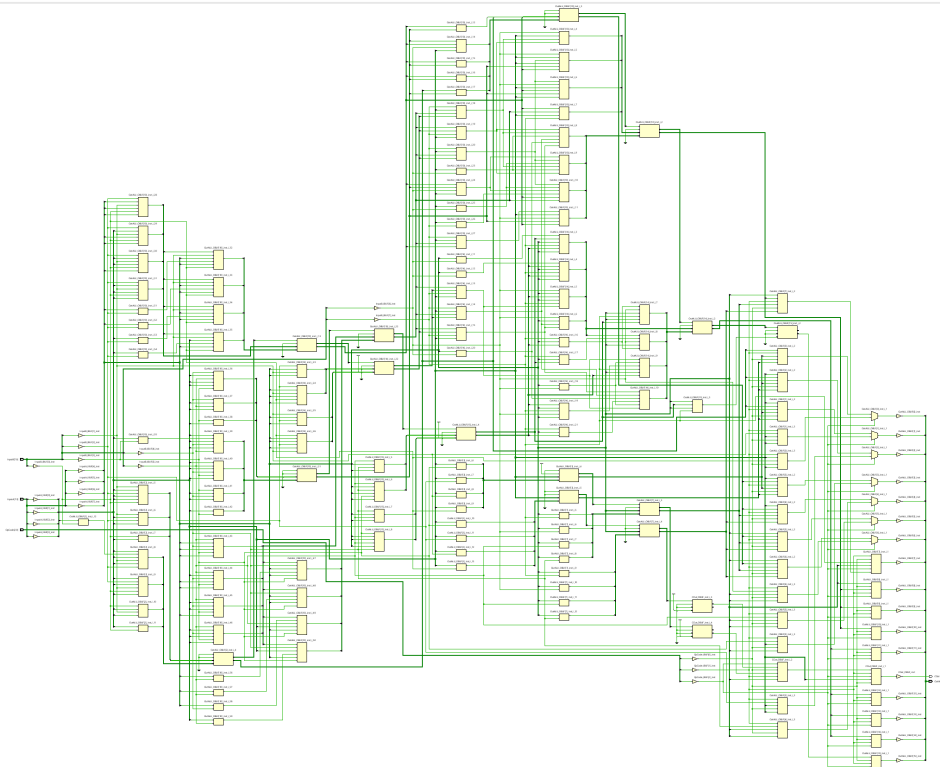


Figure 5.2: Synthesized Gate-Level Netlist

Unlike the RTL schematic, the synthesized schematic represents the actual hardware resources used during FPGA implementation. Although more complex, it accurately reflects the optimized digital circuit generated by the synthesis engine.

5.4 Synthesis Resource Utilization

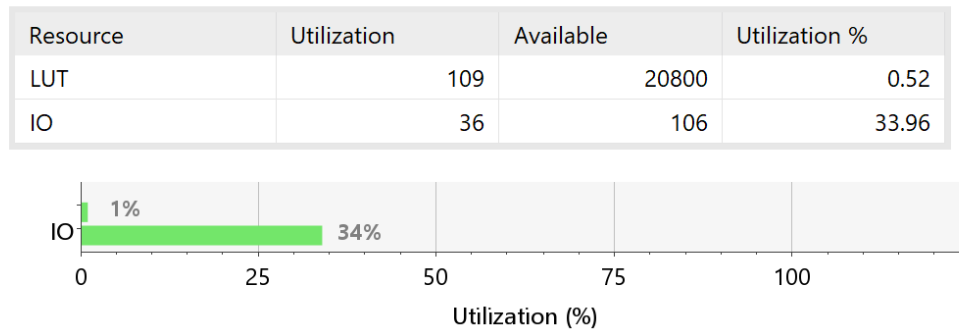
The synthesis report generated by AMD Vivado provides detailed information regarding the FPGA resources consumed by the implemented ALU. Resource utilization is an important metric for evaluating the hardware efficiency of a digital design.

Table 5.1 summarizes the utilization of the major FPGA resources after synthesis.

Table 5.1: Post-Synthesis Resource Utilization

Resource	Used	Available	Utilization (%)
LUT	109	20800	0.52
IO	36	106	33.96

Figure 5.3 presents the graphical resource utilization report generated by Vivado.

**Figure 5.3:** Post-Synthesis Resource Utilization Report

The synthesis results indicate that the proposed ALU requires only 109 Look-Up Tables (LUTs), corresponding to approximately 0.52% of the available LUT resources on the selected FPGA device. This demonstrates that the design is highly area-efficient.

The implementation utilizes 36 Input/Output (IO) pins, representing approximately 33.96% of the available IO resources. The relatively higher IO utilization is expected because the ALU exposes multiple input and output signals to the FPGA pins.

Overall, the synthesis report confirms that the proposed ALU has a very small hardware footprint while successfully implementing all required arithmetic and logical operations. The low logic utilization indicates that the design can be readily integrated into larger FPGA-based digital systems without imposing significant resource constraints.

5.5 Power Analysis

In addition to hardware resource utilization, AMD Vivado provides an estimate of the power consumed by the synthesized design. The power analysis report offers valuable insight into the expected energy requirements of the ALU before hardware implementation. These estimates are based on synthesis results and default switching activity and may vary after place-and-route or when actual input activity is available.

Figure 5.4 presents the post-synthesis power estimation generated by AMD Vivado.

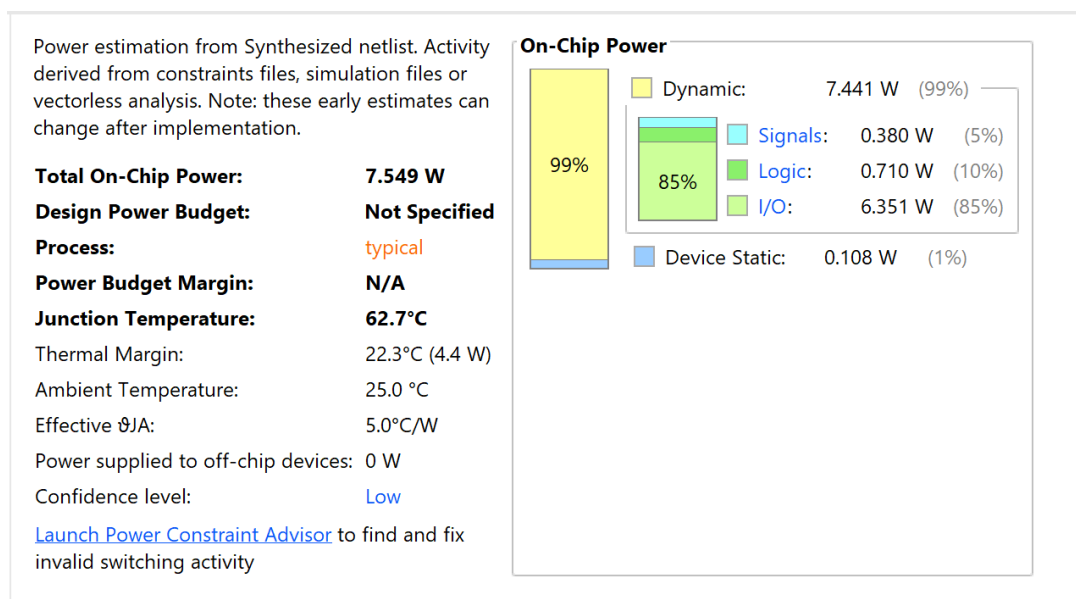


Figure 5.4: Post-Synthesis Power Estimation Report

The total estimated on-chip power consumption of the synthesized ALU is approximately **7.549 W**. Of this, the dynamic power contributes approximately **7.441 W (99%)** of the total power, while the static device power is only **0.108 W (1%)**.

The dynamic power is further divided into three major components:

- **Signals:** 0.380 W (5%)
- **Logic:** 0.710 W (10%)
- **Input/Output (I/O):** 6.351 W (85%)

The report indicates that the majority of the estimated power consumption originates from the FPGA input/output resources rather than the internal combinational logic. This is expected because the ALU exposes multiple input and output signals to the FPGA pins, while the internal logic occupies only a small fraction of the available resources.

Additional thermal information reported by Vivado indicates an estimated junction temperature of approximately **62.7 °C** under typical operating conditions, with an ambient temperature of **25 °C**. The reported thermal margin of **22.3 °C** suggests that the design operates well within acceptable thermal limits for the selected FPGA device.

It should be noted that these values represent pre-implementation estimates. More accurate power measurements can be obtained after implementation using realistic switching activity generated from post-route simulations.

5.6 Hardware Optimization

The synthesis engine automatically performs several optimizations to improve hardware efficiency and performance. These optimizations include:

- Logic minimization
- Resource sharing
- Multiplexer optimization
- Carry-chain inference for arithmetic operations
- Constant propagation
- Removal of redundant logic

These optimizations reduce the overall hardware complexity while preserving the intended functionality of the ALU.

5.7 FPGA Resources

The synthesized design consists primarily of the following FPGA resources.

Table 5.2: FPGA Resources Utilized by the ALU

Resource	Function
LUTs (Look-Up Tables)	Combinational Logic Implementation
MUXes	Opcode Selection Logic
Carry Chain	Addition and Subtraction Operations
IBUF	Input Signal Buffering
OBUF	Output Signal Buffering
Routing Resources	Signal Interconnections

5.8 Discussion

The RTL schematic confirms that the Verilog description has been translated correctly into a combinational ALU architecture. The synthesized schematic further demonstrates that the FPGA synthesis tool successfully optimized the design into hardware-specific primitives.

No synthesis errors or critical warnings were encountered during the implementation process. The successful synthesis validates that the Verilog RTL is fully synthesizable and suitable for FPGA realization.

5.9 Advantages of RTL Synthesis

RTL synthesis provides several important benefits during FPGA and ASIC design.

- Confirms synthesizability of the HDL design.
- Detects hardware design issues before implementation.
- Optimizes hardware utilization.
- Generates technology-specific hardware.
- Enables timing and resource analysis.
- Produces the gate-level implementation required for FPGA programming.

CHAPTER 6

Comparative Evaluation

6.1 Introduction

The primary objective of this project was not only to design an 8-bit Arithmetic Logic Unit (ALU) but also to investigate the effectiveness of Artificial Intelligence (AI) in assisting the hardware verification process. Although AI significantly reduced the time required to generate an initial verification environment, a detailed analysis revealed that the generated testbench required further refinement to achieve comprehensive functional verification.

This chapter presents a comparative evaluation between the AI-generated testbench, the manually improved verification environment, and the final validated testbench used for simulation.

6.2 Comparison of Testbenches

The AI-generated testbench successfully verified the mandatory functional requirements specified in the project description. However, it lacked several important verification scenarios required for robust digital design validation.

Table 6.1 compares the three verification environments.

Table 6.1: Comparison of Verification Approaches

Feature	AI Generated	Improved	Final
Mandatory Functional Tests	Yes	Yes	Yes
Output Display	Yes	Yes	Yes
Simulation Timing	Basic	Improved	Validated
Overflow Testing	No	Yes	Yes
Zero Input Verification	No	Yes	Yes
Boundary Shift Testing	No	Yes	Yes
Maximum Multiplication	No	Yes	Yes
Random Test Vectors	No	Yes	Yes
Verification Coverage	Basic	Good	Comprehensive

6.3 Analysis of the AI-Generated Testbench

The initial AI-generated testbench successfully instantiated the ALU module and verified all mandatory arithmetic and logical operations specified in the project requirements. The generated verification environment produced correct outputs for the supplied input combinations and demonstrated that AI can rapidly generate functional verification code.

However, detailed inspection revealed several limitations.

The AI-generated testbench did not include:

- Boundary condition testing
- Arithmetic overflow verification
- Carry output validation
- Maximum-value multiplication
- Shift boundary conditions
- Randomized verification
- Comprehensive functional coverage

Consequently, although the generated testbench satisfied the minimum project requirements, it was insufficient for complete functional verification.

6.4 Manual Improvements

Several modifications were introduced to improve the verification quality.

The following additional test scenarios were incorporated.

- Zero input verification
- Addition overflow
- Subtraction resulting in zero
- Maximum multiplication
- Left shift boundary condition
- Right shift boundary condition
- Maximum logical operations
- Random test vectors

These improvements significantly increased verification coverage and improved confidence in the correctness of the RTL implementation.

6.5 Prompt Refinement

One of the major observations during this project was that the quality of AI-generated code depends strongly on the quality of the prompt provided to the model.

The original prompt requested only the mandatory verification cases. After analyzing the generated testbench, the prompt was refined to explicitly include overflow conditions, boundary cases, logical verification, maximum-value testing, and random input vectors.

The refined prompt produced a substantially more comprehensive verification environment, thereby reducing the amount of manual modification required.

6.6 Lessons Learned

The project provided several valuable insights into modern digital design and verification workflows.

- AI significantly accelerates the development of verification environments.
- AI-generated code should always be reviewed before use.
- Engineering judgment remains essential during verification.
- Boundary conditions are critical for reliable digital systems.
- Prompt refinement can substantially improve AI-generated results.
- Simulation should always precede hardware synthesis.
- Functional verification is equally important as RTL design.

6.7 Overall Evaluation

The final verification environment successfully validated all implemented ALU operations under both normal and boundary operating conditions.

Behavioral simulation confirmed that the ALU correctly performed:

- Addition
- Subtraction
- Multiplication

- Left Shift
- Right Shift
- Logical AND
- Logical OR
- Logical XOR

Additional verification through overflow testing, carry validation, maximum-value operations, and randomized test vectors further demonstrated the reliability of the proposed design.

CHAPTER 7

Conclusion and Future Scope

7.1 Conclusion

This project successfully demonstrated the complete design, verification, and synthesis of an 8-bit Arithmetic Logic Unit (ALU) using Verilog Hardware Description Language (HDL). The implemented ALU performs eight arithmetic and logical operations, including addition, subtraction, multiplication, left shift, right shift, logical AND, logical OR, and logical XOR, which are selected using a 3-bit opcode.

The RTL design was implemented using synthesizable Verilog HDL and verified through behavioral simulation using AMD Vivado 2025.2. Functional verification confirmed that all implemented operations produced the expected outputs under the specified input conditions. The correctness of the design was further validated using waveform analysis and simulation console outputs generated by the XSim simulator.

A significant aspect of this project was the incorporation of Artificial Intelligence during the verification phase. An initial Verilog testbench was generated using an AI language model based on a structured prompt. The generated verification environment was subsequently analyzed to identify missing verification scenarios such as overflow conditions, boundary cases, and random test vectors. Through prompt refinement and manual engineering analysis, the verification coverage was significantly improved, resulting in a more comprehensive and reliable verification environment.

Following successful simulation, the design was synthesized using AMD Vivado. The generated RTL schematic and synthesized gate-level netlist confirmed that the Verilog implementation is fully synthesizable and suitable for FPGA realization. The synthesis process further demonstrated the translation of the RTL description into FPGA-specific hardware resources such as lookup tables (LUTs), multiplexers, carry chains, and input/output buffers.

Overall, this project provided practical experience in the complete digital design workflow, including RTL development, AI-assisted verification, behavioral simulation, debugging, synthesis, and design validation. It also highlighted the importance of combining AI-assisted automation with engineering judgment to achieve reliable hardware verification.

7.2 Future Scope

Although the implemented ALU satisfies all functional requirements specified in the project, several enhancements can be incorporated in future versions to improve its functionality and applicability.

Possible future extensions include:

- Design of parameterized ALUs supporting 8-bit, 16-bit, 32-bit, and 64-bit architectures.
- Implementation of additional arithmetic operations such as division, modulus, increment, and decrement.
- Addition of rotate-left and rotate-right instructions.
- Incorporation of comparison operations for processor applications.
- Generation of processor status flags including Zero, Sign, Overflow, and Parity flags.
- FPGA implementation on a physical development board for real-time hardware verification.
- Timing analysis and optimization for higher operating frequencies.
- Integration of the ALU into a complete processor datapath.
- Development of a constrained-random verification environment using SystemVerilog and Universal Verification Methodology (UVM).
- Exploration of advanced AI-assisted verification techniques for automatic generation of directed and constrained-random testbenches.

The knowledge gained through this project forms a strong foundation for more advanced digital design projects involving processors, FPGA-based systems, and VLSI design methodologies.

References

1. M. Morris Mano and Michael D. Ciletti, *Digital Design*, 6th Edition, Pearson Education, 2018.
2. J. Bhasker, *A Verilog HDL Primer*, 3rd Edition, Star Galaxy Publishing, 2005.
3. Stephen Brown and Zvonko Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd Edition, McGraw-Hill Education, 2019.
4. AMD, *Vivado Design Suite User Guide: Synthesis (UG901)*, 2025.
5. AMD, *Vivado Design Suite User Guide: Logic Simulation (UG900)*, 2025.
6. AMD, *Vivado Design Suite User Guide: Power Analysis and Optimization*, 2025.
7. OpenAI, *ChatGPT*, used for AI-assisted testbench generation, verification enhancement, and prompt refinement, 2026.